

Relatório de Auditoria de Segurança

TechPulse AI (tech-insights-blog)

Data	20 de agosto de 2026
Repositório	leonardodebs/tech-insights-blog
Achados	8 (2 de severidade alta)
Controles validados	16 pontos fortes com evidência

Escopo auditado

- Frontend React 19 + Vite 6 (SPA, HashRouter, base /tech-insights-blog/)
- Supabase: Postgres com RLS, Auth (senha + TOTP), 2 Edge Functions (Deno)
- Servidor Express de desenvolvimento (server.ts)
- Scripts de build/SSG e automação de conteúdo (Claude API + LinkedIn)
- 3 workflows do GitHub Actions e deploy no GitHub Pages
- Histórico completo do git e bundle JavaScript publicado em produção

Nota metodológica

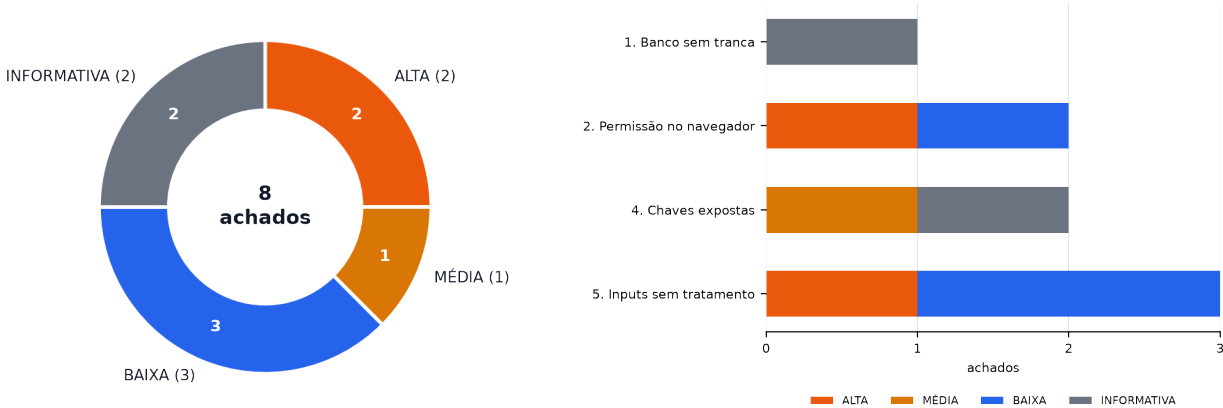
Cada categoria genérica foi mapeada para o equivalente real desta stack antes da análise, conforme abaixo. Todo achado foi confirmado no código; nada foi inferido.

- Banco sem tranca.** O mecanismo de isolamento deste projeto é o RLS do Supabase. Foram inspecionadas as policies em supabase/schema.sql, os advisors de segurança do próprio Supabase e todo acesso a tabela no código (6 pontos, uma única tabela: posts).
- Permissão no navegador.** Cada gate de papel do frontend (sessão, AAL/MFA) foi cruzado com o endpoint correspondente nas Edge Functions, verificando se o servidor repete a checagem.
- IDOR.** Enumerados TODOS os 6 handlers de rota do backend (4 no Express, 2 Edge Functions), verificando se objetos acessados por ID validam posse/autorização.
- Chaves expostas.** Varredura por padrões de segredo no código versionado, .env.example, workflows, histórico completo do git e bundle JS publicado, incluindo decodificação do JWT embutido.
- Inputs sem tratamento.** Busca por dangerouslySetInnerHTML/innerHTML/eval no frontend, revisão do pipeline de markdown e de toda geração de HTML/XML no servidor e no SSG.

Resumo executivo



A auditoria percorreu 65 arquivos versionados e cerca de 3.900 linhas de código, além do histórico completo do git e do bundle publicado em produção. Foram identificados 8 achados e confirmados 16 controles funcionando corretamente. Nenhuma falha crítica foi encontrada. Os dois achados de severidade alta são um bypass de MFA no servidor e um XSS refletido no servidor de desenvolvimento.



À esquerda, distribuição por severidade. À direita, achados por categoria auditada, empilhados por severidade.

Pontos fracos: os riscos centrais

- 1. O segundo fator protege a interface, não o servidor. Quem tem a senha do administrador alcança as duas operações privilegiadas por chamada direta à API, sem TOTP (A-01).
- 2. O servidor de desenvolvimento concentra três problemas ao mesmo tempo: XSS refletido, stack trace na resposta e escuta em todas as interfaces, enquanto carrega a chave service_role e a chave da Anthropic no ambiente (A-02, A-03, A-04).
- 3. Existe divergência entre a postura de produção e a do ambiente de desenvolvimento: correções aplicadas ao site publicado, como a remoção do picsum.photos, não foram replicadas no server.ts (A-06).

Pontos fortes verificados

Cada item abaixo foi conferido no código e está correto. A lista também serve como prova da cobertura da auditoria.

Controle	Evidência	Observação
RLS habilitado com política mínima	supabase/schema.sql:19-28	Leitura pública explícita e escrita restrita a service_role. Advisors de segurança do Supabase retornaram zero alertas.
Nenhum segredo no bundle publicado	assets/index-Dm8EKp0Q.js (produção)	O único JWT embutido decodifica para {"role":"anon"}, público por design. Zero ocorrências de service_role, chave Anthropic, token LinkedIn ou PAT.
Nenhum segredo hardcoded no repositório	.env.example e todo o código versionado	Somente placeholders. Varredura por padrões (sk-, ghp_, AKIA, Alza, chaves privadas) não encontrou nada.
Histórico do git limpo	todos os commits desde a criação	Nenhum arquivo .env ou de credencial commitado. A única correspondência suspeita (AQpD2UMJ...) foi verificada e é um hash sha512 de integridade do npm, falso positivo.
Pipeline de markdown na ordem correta	src/App.tsx:342	rehypeRaw converte o HTML bruto em AST e rehypeSanitize sanea em seguida. Inverter essa ordem foi o bug corrigido em rodada anterior; permanece correto.
SSG sanitiza o HTML pré-renderizado	scripts/generate-post-pages.ts + src/lib/sanitizeHtml.ts	A saída do marked passa pelo mesmo rehype-sanitize do SPA antes de entrar no HTML estático, com 4 testes de regressão.
Frontend sem primitivas perigosas	todo o diretório src/	Zero ocorrências de dangerouslySetInnerHTML, innerHTML, outerHTML, insertAdjacentHTML, eval, new Function ou document.write.
Edge Functions fail-closed	supabase/functions/_shared/guard.ts:58-98	Ausência de header, JWT inválido ou usuário fora da allowlist resultam em 401/403. Configuração ausente nega o acesso em vez de liberar.
CORS restrito a uma origem	supabase/functions/_shared/guard.ts:16-23	Access-Control-Allow-Origin fixo em https://leonardodebs.github.io, com Vary: Origin. Verificado em produção: origem maliciosa não é espelhada.
Identificador validado no servidor	supabase/functions/manage-posts/index.ts:15,42-44	POST_ID_PATTERN (^post-ld{10,20}\$) rejeita qualquer valor fora do formato antes de chegar ao workflow.
Workflows sem interpolação insegura	.github/workflows/manage-posts.yml:34-35,75	Inputs do usuário passam por env: e são usados como variáveis entre aspas, nunca interpolados no corpo do script. Há validação do formato no próprio workflow.
CSP sem unsafe-inline e sem unsafe-eval	index.html	default-src 'self', object-src 'none', base-uri 'self', form-action 'self'. Scripts inline foram extraídos para arquivo (public/consent-default.js).
Login sem enumeração de usuário	src/pages/AdminLogin.tsx:29	Mensagem genérica 'Email ou senha incorretos.' para qualquer falha, sem distinguir usuário inexistente de senha errada.
Credenciais sempre por variável de ambiente	src/services/automation.ts:579-580 e scripts/migrate-posts-to-supabase.ts:17-25	Nenhum fallback com valor literal. O script de migração aborta se as variáveis não estiverem definidas.
Endpoint de automação fail-closed	server.ts:43-49	Exige LOCAL_AUTOMATION_TOKEN e nega o acesso quando a variável não está configurada, em vez de liberar.
Escape correto de título e resumo	server.ts:106-107	he.encode aplicado ao conteúdo vindo do post antes da injeção nas meta tags.

Achados detalhados

Ordenados por severidade. Cada achado traz arquivo e linha exatos, o trecho real do código e a condição necessária para exploração.

ALTA**A-01 — MFA é verificado apenas no navegador; servidor não exige AAL2**

supabase/functions/_shared/guard.ts:55-101 | 2. Permissão no navegador

```
// guard.ts - requireAdmin valida assinatura e papel, mas NUNCA o AAL
const { data, error } = await supabase.auth.getUser();
const role = (user.app_metadata)?.role;
const hasAdminRole = role === "admin";
const inAllowlist = allowlist.includes(email);
if (!hasAdminRole && !inAllowlist) return 403; // <-- nenhuma checagem de aal2
```

Por que é explorável. O gate de MFA existe só no cliente: AdminPage.tsx bloqueia a interface quando a sessão está em aal1, mas as Edge Functions aceitam qualquer JWT válido de um e-mail admin, independentemente de o segundo fator ter sido cumprido. Um atacante de posse apenas da senha obtém um token aal1 direto na API de auth e chama manage-posts ou trigger-blog-post, excluindo posts ou queimando crédito da Anthropic sem nunca digitar o código TOTP. Isso anula o controle de MFA implantado na rodada anterior.

Impacto. Bypass completo do segundo fator nas duas operações privilegiadas do sistema.

Condição. Requer conhecimento da senha do admin. Não requer o dispositivo TOTP.

Correção. Em requireAdmin, ler a claim aal do JWT (ou chamar getAuthenticatorAssuranceLevel) e recusar com 403 quando o usuário possui fator TOTP verificado e a sessão está em aal1.

Também relevante: src/lib/useMfa.ts:33-39, src/pages/AdminPage.tsx:60-62

ALTA**A-02 — XSS refletido: cabeçalho Host e URL injetados sem escape nas meta tags**

server.ts:114,117 | 5. Inputs sem tratamento

```
// server.ts:106-107 - title e excerpt SÃO escapados
const safeTitle = he.encode(post.title);
const safeExcerpt = he.encode(post.excerpt);

// server.ts:114 e 117 - host e URL entram CRUS no atributo content
html = html.replace(/<meta property="og:url" content=".*?" \>/,
  `<meta property="og:url" content="${req.protocol}://${req.get('host')}${req.originalUrl}" />`
);
```

Por que é explorável. req.get('host') e req.originalUrl são controlados por quem faz a requisição e vão direto para dentro de um atributo HTML sem escape. Basta um Host contendo aspas para escapar do atributo e injetar script na página. O contraste com as linhas 106-107, onde he.encode é corretamente aplicado, mostra que o escape foi esquecido nestas duas. Confirmado que index.html:30 e :37 contêm exatamente as tags que a regex substitui, portanto a injeção encontra alvo.

Impacto. Execução de JavaScript no contexto da página, com roubo da sessão do admin (guardada em localStorage).

Condição. server.ts é o servidor de desenvolvimento (npm run dev), não é publicado no GitHub Pages. A exposição real vem de A-03: o processo escuta em 0.0.0.0.

Correção. Aplicar he.encode também na URL, ou montar a URL canônica a partir de uma constante do próprio servidor em vez de refletir o Host enviado pelo cliente.

Também relevante: index.html:30,37 (alvo da substituição)

MÉDIA

A-03 — Servidor de desenvolvimento escuta em todas as interfaces com credenciais de produção no ambiente

server.ts:149 | 4. Chaves expostas

```
app.listen(PORT, "0.0.0.0", () => {  
  console.log(`Server running on http://localhost:${PORT}`);  
});
```

Por que é explorável. O bind em 0.0.0.0 publica o servidor para toda a rede local, não apenas para a máquina do desenvolvedor. Esse processo carrega SUPABASE_SERVICE_ROLE_KEY e ANTHROPIC_API_KEY no ambiente, e é o mesmo que expõe o XSS de A-03. Qualquer dispositivo na mesma rede alcança as rotas.

Impacto. Amplia o alcance do XSS (A-02) e do vazamento de stack trace (A-04) de local para toda a rede.

Condição. Vale enquanto o servidor estiver rodando localmente (npm run dev).

Correção. Trocar o bind para 127.0.0.1, que é o correto para um servidor de desenvolvimento.

Também relevante: server.ts:43-49, src/services/automation.ts:579-580

BAIXA

A-04 — Stack trace devolvido ao cliente nas respostas de erro

server.ts:72 | 5. Inputs sem tratamento

```
res.status(500).json({  
  success: false,  
  error: error.message || "Unknown error during automation",  
  stack: process.env.NODE_ENV !== 'production' ? error.stack : undefined  
});
```

Por que é explorável. Sempre que NODE_ENV não for exatamente 'production', o stack trace completo volta no corpo da resposta, revelando caminhos absolutos do sistema de arquivos e a estrutura interna do projeto. A condição é frágil: basta a variável não estar definida.

Impacto. Divulgação de informação que facilita o mapeamento do ambiente por um atacante.

Condição. Combinado com A-03, fica acessível a qualquer um na rede local.

Correção. Registrar o stack apenas no log do servidor e nunca no corpo da resposta HTTP.

BAIXA

A-05 — Painel administrativo liberado por existência de sessão, não por papel de admin

src/pages/AdminPage.tsx:51 | 2. Permissão no navegador

```
// Sem sessão -> login por senha.  
if (!session) {  
  return <AdminLogin onLogin={() => refreshMfa()} />;  
}  
// ...qualquer usuário autenticado prossegue para o painel
```

Por que é explorável. A condição verifica apenas se existe sessão, não se o usuário é administrador. Qualquer conta autenticada no projeto Supabase renderiza a interface completa do painel, incluindo os botões de gerar e excluir post. O impacto é contido porque as ações são bloqueadas no servidor com 403 e a lista de posts já é pública, mas a interface sugere um privilégio que o usuário não tem.

Impacto. Exposição de interface administrativa a usuários não privilegiados. Sem perda de dados.

Condição. Só é alcançável se o cadastro público estiver habilitado no projeto Supabase.

Correção. Verificar o papel do usuário (app_metadata.role ou allowlist) antes de renderizar o painel, mantendo a checagem no servidor como autoridade final.

Também relevante: src/pages/AdminPanel.tsx:51-65

BAIXA

A-06 — Imagem de terceiro (picsum.photos) reintroduzida no servidor de desenvolvimento

server.ts:120-122 | 5. Inputs sem tratamento

```
const imageUrl = `https://picsum.photos/seed/${post.id}/1200/630`;
html = html.replace(/<meta property="og:image" content=".*?" \>/,
  `<meta property="og:image" content="${imageUrl}" />`);
```

Por que é explorável. A auditoria anterior removeu o picsum.photos da produção justamente por entregar o IP do visitante a um terceiro sem base legal. O servidor de desenvolvimento manteve a referência, criando divergência entre o que a política de privacidade declara e o que este código faz.

Impacto. Vazamento de IP a terceiro e inconsistência com a política de privacidade publicada.

Condição. Só afeta quem acessar via servidor de desenvolvimento.

Correção. Apontar para a imagem própria em public/og/default.png, igual à produção.

INFORMATIVA

A-07 — Comparação de token sem tempo constante

server.ts:44 | 4. Chaves expostas

```
const expected = process.env.LOCAL_AUTOMATION_TOKEN;
if (!expected || req.get("x-automation-token") !== expected) {
```

Por que é explorável. O operador !== interrompe na primeira diferença, o que teoricamente permite inferir o token byte a byte medindo o tempo de resposta. Na prática o ruído de rede num servidor local torna o ataque impraticável, e a guarda está corretamente fail-closed.

Impacto. Teórico. Registrado por completude.

Condição. Requer milhares de requisições e rede de latência estável.

Correção. Usar crypto.timingSafeEqual sobre buffers de mesmo tamanho.

INFORMATIVA

A-08 — Estado do RLS em produção não pôde ser confirmado nesta execução

supabase/schema.sql:19-28 | 1. Banco sem tranca

```
ALTER TABLE posts ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Public read" ON posts FOR SELECT USING (true);
CREATE POLICY "Service write" ON posts FOR ALL USING (auth.role() = 'service_role');
```

Por que é explorável. O schema versionado está correto. A verificação direta contra o banco em produção falhou porque o DNS da rede local (servidor homelab, 192.168.100.2) responde 'domínio inexistente' para supabase.co, bloqueando tanto o SQL quanto o teste de escrita com a chave anon. Registrado como lacuna e não como falha porque os advisors de segurança do Supabase retornaram zero alertas, e eles sinalizam exatamente tabela pública sem RLS. O precedente do achado B-01 da rodada anterior (função em produção divergente do repositório) justifica não assumir equivalência sem provar.

Impacto. Nenhum impacto conhecido. É uma lacuna de cobertura da auditoria.

Condição. Depende apenas de resolver o DNS local para supabase.co.

Correção. Repetir com DNS funcional: SELECT sobre pg_policies e tentativa de INSERT/UPDATE/DELETE com a chave anon, que deve ser recusada.

Tabela de achados por categoria

1. Banco sem tranca

Sev.	Arquivo:linha	Descrição
INFORMATIVA	supabase/schema.sql:19-28	A-08 — Estado do RLS em produção não pôde ser confirmado nesta execução

2. Permissão no navegador

Sev.	Arquivo:linha	Descrição
ALTA	supabase/functions/_shared/guard.ts:55-101	A-01 — MFA é verificado apenas no navegador; servidor não exige AAL2
BAIXA	src/pages/AdminPage.tsx:51	A-05 — Pannel administrativo liberado por existência de sessão, não por papel de admin

4. Chaves expostas

Sev.	Arquivo:linha	Descrição
MÉDIA	server.ts:149	A-03 — Servidor de desenvolvimento escuta em todas as interfaces com credenciais de produção no ambiente
INFORMATIVA	server.ts:44	A-07 — Comparação de token sem tempo constante

5. Inputs sem tratamento

Sev.	Arquivo:linha	Descrição
ALTA	server.ts:114, 117	A-02 — XSS refletido: cabeçalho Host e URL injetados sem escape nas meta tags
BAIXA	server.ts:72	A-04 — Stack trace devolvido ao cliente nas respostas de erro
BAIXA	server.ts:120-122	A-06 — Imagem de terceiro (picsum.photos) reintroduzida no servidor de desenvolvimento

Recomendações priorizadas

Pri.	Ação	Detalhe	Achados
P1	Exigir AAL2 no servidor	Adicionar a verificação de Assurance Level em requireAdmin, recusando sessões aal1 quando houver fator TOTP verificado. Sem isso o MFA continua sendo apenas cosmético para a API.	A-01
P1	Escapar host e URL no server.ts	Aplicar he.encode nas linhas 114 e 117, ou usar uma URL canônica constante em vez de refletir o cabeçalho Host.	A-02
P2	Restringir o servidor de desenvolvimento ao localhost	Trocar o bind de 0.0.0.0 para 127.0.0.1 e remover o stack trace do corpo da resposta.	A-03, A-04
P2	Alinhar o dev server à produção	Substituir o picsum.photos pela imagem própria já usada no site publicado.	A-06
P3	Verificar papel antes de renderizar o painel	Checar admin no cliente para não exibir interface privilegiada a quem não tem permissão, mantendo o servidor como autoridade.	A-05
P3	Fechas as lacunas de verificação	Repetir o teste de RLS contra o banco com DNS funcional e adotar comparação em tempo constante para o token local.	A-07, A-08

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só para não gerar ruído.

--- ISSUE 1 ---

Título: [Segurança] MFA é verificado apenas no navegador; Edge Functions aceitam sessão aal1

Labels: security, alta

Problema

O gate de MFA existe somente no cliente. `AdminPage.tsx` bloqueia a interface quando a sessão está em `aal1`, mas `requireAdmin` nas Edge Functions valida apenas a assinatura do JWT e o papel de administrador. O Assurance Level nunca é consultado.

Na prática, quem tem a senha do administrador obtém um token `aal1` direto na API de autenticação e chama `manage-posts` ou `trigger-blog-post` sem nunca digitar o código TOTP. Isso anula o controle de MFA implantado na rodada anterior da auditoria.

Evidência

`supabase/functions/\_shared/guard.ts:55-101`

```

` ` `ts
const { data, error } = await supabase.auth.getUser();
if (error || !data?.user) return { error: jsonResponse({ error: "Não autorizado." }, 401) };

const role = (user.app_metadata as Record<string, unknown> | null)?.role;
const hasAdminRole = role === "admin";
const inAllowlist = email.length > 0 && allowlist.includes(email);

if (!hasAdminRole && !inAllowlist) {
  return { error: jsonResponse({ error: "Acesso negado." }, 403) };
}
// nenhuma verificação de aal2 em nenhum ponto do arquivo
` ` `

```

Contraste com o gate do cliente, que faz a checagem corretamente:

`src/lib/useMfa.ts:33-39` e `src/pages/AdminPage.tsx:60-62`

```

` ` `tsx
if (mfa.state === 'needs-challenge') {
  return <MfaChallenge onVerified={() => refreshMfa()} onCancel={handleLogout} />;
}
` ` `

```

Impacto

Bypass completo do segundo fator nas duas operações privilegiadas: exclusão de posts e disparo da geração (que consome crédito da API da Anthropic).

Sugestão de correção

Em `requireAdmin`, ler a claim `aal` do JWT do chamador e recusar com 403 quando o usuário possuir fator TOTP verificado e a sessão estiver em `aal1`.

```

` ` `ts
// após validar o usuário
const { data: aalData } = await supabase.auth.mfa.getAuthenticatorAssuranceLevel();
if (aalData?.nextLevel === 'aal2' && aalData?.currentLevel !== 'aal2') {
  return { error: jsonResponse({ error: "Segundo fator obrigatório." }, 403) };
}
` ` `

```

Critérios de aceite

- [] `requireAdmin` recusa com 403 uma sessão `aal1` de usuário com fator TOTP verificado
- [] `requireAdmin` continua aceitando sessão `aal2` de administrador
- [] Usuário sem fator cadastrado continua funcionando (não quebra o fluxo atual)
- [] Teste automatizado cobrindo os três casos acima
- [] Verificado em produção: token `aal1` recebe 403 em `manage-posts`

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] XSS refletido no server.ts: cabeçalho Host e URL entram sem escape nas meta tags**Labels:** security, alta

Problema

Ao injetar as meta tags de compartilhamento, `server.ts` escapa corretamente o título e o resumo do post, mas insere `req.get('host')` e `req.originalUrl` **sem nenhum tratamento** dentro de um atributo HTML. Ambos são controlados por quem faz a requisição.

Um `Host` contendo aspas escapa do atributo `content` e injeta markup arbitrário na página.

Evidência

`server.ts:106-107` (o escape correto, para comparação):

```
``ts
const safeTitle = he.encode(post.title);
const safeExcerpt = he.encode(post.excerpt);
``
```

`server.ts:114` e `server.ts:117` (sem escape):

```
``ts
html = html.replace(/<meta property="og:url" content=".*?" \/>/,
  '<meta property="og:url" content="${req.protocol}://${req.get('host')}${req.originalUrl}" />`);
;

html = html.replace(/<meta property="twitter:url" content=".*?" \/>/,
  '<meta property="twitter:url" content="${req.protocol}://${req.get('host')}${req.originalUrl}" />`);
``
```

Confirmado que `index.html:30` e `index.html:37` contêm exatamente as tags que a regex substitui, portanto a injeção encontra alvo.

Impacto

Execução de JavaScript no contexto da página. Como a sessão do Supabase fica em `localStorage`, um administrador que abrisse uma URL manipulada teria o token roubado.

Condição de explorabilidade

`server.ts` é o servidor de desenvolvimento (`npm run dev`) e não é publicado no GitHub Pages. A exposição real vem de o processo escutar em `0.0.0.0` (ver issue 3), o que o torna alcançável por toda a rede local.

Sugestão de correção

Escapar a URL da mesma forma que o título, ou preferencialmente montar a URL canônica a partir de uma constante do servidor em vez de refletir o `Host` do cliente.

```
``ts
const canonical = he.encode(`${BASE_URL}${req.originalUrl}`);
``
```

Critérios de aceite

- [] `Host` e `originalUrl` não chegam ao HTML sem escape
- [] Requisição com `Host: x"><script>alert(1)</script>` não produz markup executável
- [] Título e resumo continuam escapados como já estão
- [] Teste cobrindo a tentativa de quebra de atributo

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Endurecer o servidor de desenvolvimento: bind, stack trace e imagem de terceiro

Labels: security, media

Problema

Três problemas do mesmo arquivo, agrupados por serem do mesmo tema e terem a mesma correção de contexto.

****1. Escuta em todas as interfaces.**** O servidor publica-se para toda a rede local, não apenas para a máquina do desenvolvedor, enquanto carrega `SUPABASE\_SERVICE\_ROLE\_KEY` e `ANTHROPIC\_API\_KEY` no ambiente.

****2. Stack trace na resposta HTTP.**** Sempre que `NODE\_ENV` não for exatamente `production`, o stack completo volta no corpo, revelando caminhos absolutos e a estrutura do projeto. A condição é frágil: basta a variável não estar definida.

****3. Imagem de terceiro reintroduzida.**** A auditoria anterior removeu o `picsum.photos` da produção por entregar o IP do visitante a um terceiro sem base legal. O servidor de desenvolvimento manteve a referência.

Evidência

`server.ts:149`

```
```ts
app.listen(PORT, "0.0.0.0", () => {
 console.log(`Server running on http://localhost:${PORT}`);
});
```
```

`server.ts:72`

```
```ts
res.status(500).json({
 success: false,
 error: error.message || "Unknown error during automation",
 stack: process.env.NODE_ENV !== 'production' ? error.stack : undefined
});
```
```

`server.ts:120-122`

```
```ts
const imageUrl = `https://picsum.photos/seed/${post.id}/1200/630`;
```
```

Impacto

O bind aberto transforma problemas locais em problemas de rede: amplia o alcance do XSS da issue 2 e do vazamento de stack trace de uma máquina para todo o segmento. A imagem de terceiro cria divergência com a política de privacidade publicada.

Sugestão de correção

```
```ts
app.listen(PORT, "127.0.0.1", () => { ... }); // 1
console.error(error); // 2: log no servidor, nunca na resposta
const imageUrl = `${BASE_URL}/og/default.png`; // 3: mesma imagem da produção
```
```

Critérios de aceite

- [] Servidor responde em `127.0.0.1:3000` e não responde pelo IP de rede da máquina
- [] Resposta de erro 500 não contém o campo `stack` em nenhum valor de `NODE\_ENV`
- [] Nenhuma referência a `picsum.photos` permanece no repositório
- [] `grep -rn "picsum" .` retorna vazio

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Painel admin renderiza para qualquer usuário autenticado, sem checar papel**Labels:** security, baixa**## Problema**

O gate do painel verifica apenas se existe sessão, não se o usuário é administrador. Qualquer conta autenticada no projeto Supabase renderiza a interface completa, incluindo os botões de gerar e excluir post.

O impacto é contido porque o servidor bloqueia as ações com 403 e a lista de posts já é pública, mas a interface promete um privilégio que o usuário não tem, e isso é uma pista útil para quem estiver sondando o sistema.

Evidência

```
`src/pages/AdminPage.tsx:51`
```

```
````tsx
// Sem sessão -> login por senha.
if (!session) {
 return <AdminLogin onLogin={() => refreshMfa()} />;
}
// qualquer usuário autenticado prossegue para o painel
````
```

Impacto

Exposição de interface administrativa a usuários não privilegiados. Sem perda ou alteração de dados, já que a autorização do servidor permanece efetiva.

Condição de explorabilidade

Só é alcançável se o cadastro público estiver habilitado no projeto Supabase.

Sugestão de correção

Consultar o papel do usuário antes de renderizar `AdminPanel`, exibindo uma tela de acesso negado caso contrário. A checagem do servidor continua sendo a autoridade final; esta é apenas para a coerência de interface.

Critérios de aceite

- [] Usuário autenticado sem papel de admin recebe tela de acesso negado, não o painel
- [] Administrador continua acessando normalmente
- [] A verificação do servidor permanece intacta (não é substituída pela do cliente)
- [] Confirmado que o cadastro público está desabilitado no projeto Supabase

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Fechar lacunas de verificação: RLS em produção e comparação de token

Labels: security, informativa

Problema

Dois itens menores levantados durante a auditoria, agrupados por serem ambos de baixo risco e de natureza verificatória.

****1. RLS de produção não confirmado.**** O schema versionado está correto, mas a verificação direta contra o banco falhou porque o DNS da rede local respondeu "domínio inexistente" para `supabase.co`. Dado o precedente do achado B-01 da rodada anterior, em que uma Edge Function em produção divergia do repositório por meses, não se deve assumir equivalência sem provar.

****2. Comparação de token sem tempo constante.**** O operador `!==` interrompe na primeira diferença. Na prática o ruído de rede torna o ataque impraticável num servidor local, mas a correção é trivial.

Evidência

`supabase/schema.sql:19-28`

```
```sql
ALTER TABLE posts ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Public read" ON posts FOR SELECT USING (true);
CREATE POLICY "Service write" ON posts FOR ALL USING (auth.role() = 'service_role');
```

`server.ts:44`

```
```ts
if (!expected || req.get("x-automation-token") !== expected) {
```

Impacto

Nenhum impacto conhecido. O primeiro item é lacuna de cobertura da auditoria; o segundo é teórico.

Sugestão de correção

Para o RLS, executar com DNS funcional:

```
```bash
deve retornar rowsecurity = true
SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname='public';

com a chave anon, todas devem ser recusadas
curl -X POST -H "apikey: $ANON" -H "Authorization: Bearer $ANON" -d '{"id":"teste"}' "$URL/rest/v1/posts"
```

Para o token, usar `crypto.timingSafeEqual` sobre buffers de mesmo tamanho.

## ## Critérios de aceite

- [ ] Confirmado por consulta que todas as tabelas públicas têm RLS habilitado
- [ ] Confirmado que INSERT, UPDATE e DELETE com a chave anon são recusados
- [ ] Comparação do token passa a usar tempo constante
- [ ] Resultado da verificação registrado neste repositório

--- FIM ISSUE 5 ---